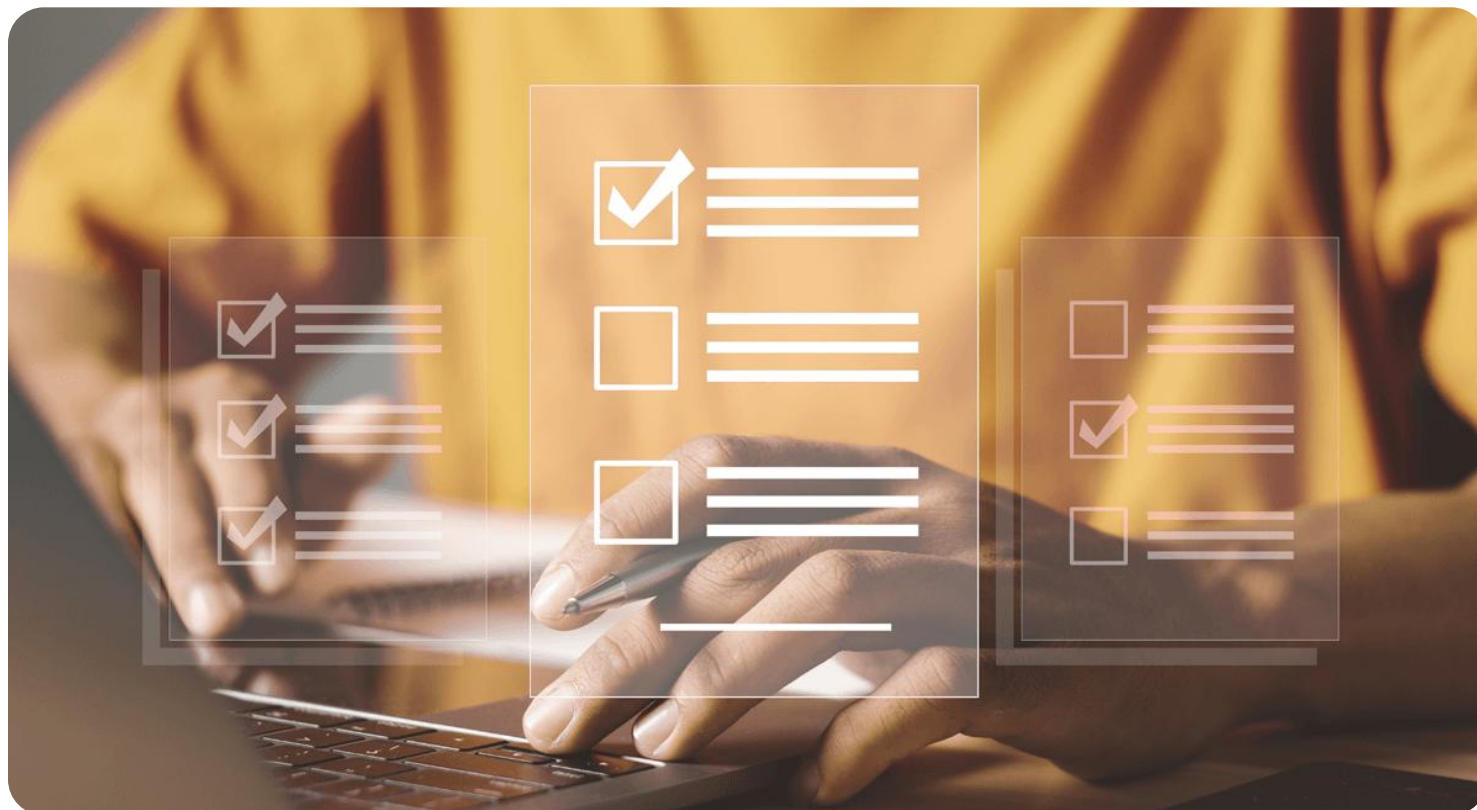




Semana 3

Desarrollo Backend III (PBY2203)

Formato de respuesta

Nombre estudiante:	Pedro Breit
Asignatura: Desarrollo Backend III	Carrera: Analista programador computacional
Profesor: Alonso Esteban Castillo Pizarro	Fecha: 31/08/2026

Contenido

Evidencia 1	3
Evidencia 2	3
Evidencia 3	3
Evidencia 4	4
Evidencia 5	4
Evidencia 6	4
Evidencia 7	5
Evidencia 8	5
Evidencia 9	6
Evidencia 10	6
Evidencia 11	7
Evidencia 12	7
Evidencia 13	8
Evidencia 14	9
Evidencia 15	10
Evidencia 16	10

Evidencia 1

```
PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_S3_Pedro_Breit\banco-batch> .\mvnw.cmd spring-boot:run "-Dspring-boot.run.arguments=-spring.batch.job.name=transaccionesJob --banco.batch.threads=3 --banco.batch.chunk-size=5 run.id=$runId" 2>& |
PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_S3_Pedro_Breit\banco-batch> >> Select-String "STEP FINALIZADO|Job:.*completed"

2026-08-28T15:40:22.851-04:00 INFO 33348 --- [banco-batch] [main] c.d.b.listener.BatchStepListener : STEP FINALIZADO | step-procesarTransaccionesStep | status=COMPLETED | leidos=1000 | escritos=482 | omitidosLectura=0 | omitidosProceso=518 | omitidosEscritura=0 | commits=200 | rollbacks=0 | duracionMs=873
2026-08-28T15:40:23.078-04:00 INFO 33348 --- [banco-batch] [main] c.d.b.listener.BatchStepListener : STEP FINALIZADO | step-generarResumenTransaccionesStep | status=COMPLETED | leidos=265 | escritos=265 | omitidosLectura=0 | omitidosProceso=0 | omitidosEscritura=0 | commits=53 | rollbacks=0 | duracionMs=211
2026-08-28T15:40:23.092-04:00 INFO 33348 --- [banco-batch] [main] o.s.b.c.l.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=transaccionesJob]] completed with the following parameters: [(JobParameter{name='run.id', value=1787945993465, type=class java.lang.String, identifying=true})] and the following status: [COMPLETED] in 1s131ms
```

Ejecución del Job de transacciones mediante procesamiento multihilo con 3 threads.

Evidencia 2

```
PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_S3_Pedro_Breit\banco-batch> docker exec banco-postgres psql -U postgres -d banco_batch -c "SELECT COUNT(*) AS total_transacciones FROM transacciones_procesadas; SELECT COUNT(*) AS total_resu-
menes FROM resumen_transacciones_diarias;"
total_transacciones
-----
482
(1 row)

total_resumenes
-----
265
(1 row)

PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_S3_Pedro_Breit\banco-batch> |
```

Validación de los resultados generados por transaccionesJob y almacenados en PostgreSQL.

Evidencia 3

```
PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_S3_Pedro_Breit\banco-batch> $runId = [DateTimeOffset]::UtcNow.ToUnixTimeMilliseconds()
PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_S3_Pedro_Breit\banco-batch> PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_S3_Pedro_Breit\banco-batch> .\mvnw.cmd spring-boot:run "-Dspring-boot.run.arguments=-spring.batch.job.name=InteresesJob --banco.batch.threads=1 --banco.batch.chunk-size=5 run.id=$runId" 2>& |
PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_S3_Pedro_Breit\banco-batch> >> Select-String "STEP FINALIZADO|Job:.*completed"

2026-08-28T15:48:35.798-04:00 INFO 26336 --- [banco-batch] [main] c.d.b.listener.BatchStepListener : STEP FINALIZADO | step-procesarInteresesStep | status=COMPLETED | leidos=1000 | escritos=370 | omitidosLectura=0 | omitidosProceso=630 | omitidosEscritura=0 | commits=200 | rollbacks=0 | duracionMs=964
2026-08-28T15:48:35.814-04:00 INFO 26336 --- [banco-batch] [main] o.s.b.c.l.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=InteresesJob]] completed with the following parameters: [(JobParameter{name='run.id', value=1787946586296, type=class java.lang.String, identifying=true})] and the following status: [COMPLETED] in 995ms
```

Ejecución del Job de cálculo de intereses con la configuración de 1 thread.

Evidencia 4

```
PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_S3_Pedro_Breit\banco-batch> docker exec banco-postgres psql -U postgres -d banco_batch -c "SELECT COUNT(*) AS cuentas_actualizadas FROM intereses_calculados; SELECT cuenta_id, nombre, saldo_inicial, tipo, tasa, interes, saldo_final FROM intereses_calculados ORDER BY cuenta_id LIMIT 10;"
cuentas_actualizadas
-----
50
(1 row)

cuenta_id | nombre      | saldo_inicial | tipo   | tasa | interes | saldo_final
-----
101 | Charlie Green | 8000.00 | prestamo | 0.0200 | 160.00 | 8160.00
102 | Diana Prince | 12000.00 | prestamo | 0.0200 | 240.00 | 12240.00
103 | Bob Johnson | 7000.00 | ahorro | 0.0100 | 70.00 | 7070.00
104 | Alice Brown | 7000.00 | prestamo | 0.0200 | 140.00 | 7140.00
105 | Steve Rogers | 10000.00 | ahorro | 0.0100 | 100.00 | 10100.00
106 | John Doe | 5000.00 | ahorro | 0.0100 | 50.00 | 5050.00
107 | Diana Prince | 8000.00 | prestamo | 0.0200 | 160.00 | 8160.00
108 | Diana Prince | 10000.00 | ahorro | 0.0100 | 100.00 | 10100.00
109 | Alice Brown | 12000.00 | prestamo | 0.0200 | 240.00 | 12240.00
110 | Charlie Green | 10000.00 | prestamo | 0.0200 | 200.00 | 10200.00
(10 rows)
```

Resultados del cálculo de intereses y actualización de saldos finales almacenados en PostgreSQL.

Evidencia 5

```
PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_S3_Pedro_Breit\banco-batch> $runId = [DateTimeOffset]::UtcNow.ToUnixTimeMilliseconds()
PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_S3_Pedro_Breit\banco-batch>
PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_S3_Pedro_Breit\banco-batch> .\mvnw.cmd spring-boot:run "-Dspring-boot.run.arguments=---spring.batch.job.name=estadosAnualesJob --banco.batch.threads=5 --banco.batch.chunk-size=5 run.id=$runId"
2026-08-28T15:56:51.182-04:00 INFO 32684 --- [banco-batch] [main] c.d.b.listener.BatchStepListener : STEP FINALIZADO | step=procesarMovimientosAnualesStep | status=COMPLETED | leidos=1000 | escritos=898 | omitidosLectura=0 | omitidosProceso=182 | omitidosEscritura=0 | commits=200 | rollbacks=0 | duracionMs=954
2026-08-28T15:56:51.236-04:00 INFO 32684 --- [banco-batch] [main] c.d.b.listener.BatchStepListener : STEP FINALIZADO | step=generarResumenAnualStep | status=COMPLETED | leidos=20 | escritos=20 | omitidosLectura=0 | omitidosProceso=0 | omitidosEscritura=0 | commits=4 | rollbacks=0 | duracionMs=37
2026-08-28T15:56:51.252-04:00 INFO 32684 --- [banco-batch] [main] o.s.b.c.l.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=estadosAnualesJob]] completed with the following parameters: [[JobParameter[name='run.id', value=1787947884690, type=class java.lang.String, identifying=true]]] and the following status: [COMPLETED] in 1542ms
```

Ejecución del Job de estados anuales utilizando procesamiento multihilo con 5 threads.

Evidencia 6

```
PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_S3_Pedro_Breit\banco-batch> docker exec banco-postgres psql -U postgres -d banco_batch -c "SELECT COUNT(*) AS total_movimientos FROM movimientos_anuales; SELECT COUNT(*) AS total_estados FROM estados_cuenta_anuales; SELECT * FROM estados_cuenta_anuales ORDER BY cuenta_id LIMIT 10;"
total_movimientos
-----
898
(1 row)

total_estados
-----
20
(1 row)

cuenta_id | total_depositos | total_retiros | total_compras | saldo_neto | cantidad_movimientos
-----
101 | 38000.00 | -10000.00 | -12700.00 | 14500.00 | 41
102 | 24200.00 | -24400.00 | -23800.00 | -24000.00 | 47
103 | 12200.00 | -15500.00 | -42400.00 | -45700.00 | 37
104 | 17300.00 | -25800.00 | -28200.00 | -36700.00 | 49
105 | 27100.00 | -26200.00 | -19400.00 | -18500.00 | 52
106 | 30300.00 | -17900.00 | -40200.00 | -26900.00 | 55
107 | 28900.00 | -30200.00 | -23100.00 | -24400.00 | 51
108 | 18200.00 | -36300.00 | -22700.00 | -40800.00 | 40
109 | 40100.00 | -28600.00 | -14700.00 | -3200.00 | 46
110 | 18600.00 | -17900.00 | -20500.00 | -19800.00 | 41
(10 rows)
```

Validación de movimientos procesados y estados de cuenta anuales generados en PostgreSQL.

Evidencia 7

```
PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_53_Pedro_Breit\banco-batch> Get-Content .\evidencia_skip.txt |
>> Select-String "SKIP POLICY/REGISTRO OMITIDO" |
>> Select-Object -First 6

2026-08-28T16:04:04.697-04:00 WARN 6952 --- [banco-batch] [transacciones-3] c.d.bancobatch.policy.BancoSkipPolicy : SKIP POLICY | registro omitido=1 de 1000 | excepcion=DatoInvalidoException | motivo=Tipo de transaccion invalido: invalid
2026-08-28T16:04:04.697-04:00 WARN 6952 --- [banco-batch] [transacciones-2] c.d.bancobatch.policy.BancoSkipPolicy : SKIP POLICY | registro omitido=1 de 1000 | excepcion=DatoInvalidoException | motivo=Monto nulo en transaccion ID 4
2026-08-28T16:04:04.697-04:00 WARN 6952 --- [banco-batch] [transacciones-3] c.d.b.listener.BatchSkipListener : REGISTRO OMITIDO | etapa=PROCESS | registro=TransaccionCsv[id=3, fecha='2024-04-09', monto=800, tipo='invalid'] | motivo=Tipo de transaccion invalido: invalid
2026-08-28T16:04:04.697-04:00 WARN 6952 --- [banco-batch] [transacciones-2] c.d.b.listener.BatchSkipListener : REGISTRO OMITIDO | etapa=PROCESS | registro=TransaccionCsv[id=4, fecha='04/05/2024', monto=null, tipo='invalid'] | motivo=Monto nulo en transaccion ID 4
2026-08-28T16:04:04.715-04:00 WARN 6952 --- [banco-batch] [transacciones-2] c.d.bancobatch.policy.BancoSkipPolicy : SKIP POLICY | registro omitido=3 de 1000 | excepcion=DatoInvalidoException | motivo=Formato de fecha invalido: 2024-13-01
2026-08-28T16:04:04.715-04:00 WARN 6952 --- [banco-batch] [transacciones-2] c.d.b.listener.BatchSkipListener : REGISTRO OMITIDO | etapa=PROCESS | registro=TransaccionCsv[id=7, fecha='2024-13-01', monto=700, tipo='debito'] | motivo=Formato de fecha invalido: 2024-13-01

PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_53_Pedro_Breit\banco-batch>
PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_53_Pedro_Breit\banco-batch> Get-Content .\evidencia_skip.txt |
>> Select-String "STEP FINALIZADO|Job:.*completed" |
>> Select-Object -Last 3

2026-08-28T16:04:05.925-04:00 INFO 6952 --- [banco-batch] [ main] c.d.b.listener.BatchStepListener : STEP FINALIZADO | step=procesarTransaccionesStep | status=COMPLETED | leidos=1000 | escritos=482 | omitidosLectura=0 | omitidosProceso=518 | omitidosEscritura=0 | commits=200 | rollbacks=0 | duracionMs=1243
2026-08-28T16:04:06.167-04:00 INFO 6952 --- [banco-batch] [ main] c.d.b.listener.BatchStepListener : STEP FINALIZADO | step=generarResumenTransaccionesStep | status=COMPLETED | leidos=265 | escritos=265 | omitidosLectura=0 | omitidosProceso=0 | omitidosEscritura=0 | commits=53 | rollbacks=0 | duracionMs=223
2026-08-28T16:04:06.180-04:00 INFO 6952 --- [banco-batch] [ main] o.s.b.c.l.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=transaccionesJob]] completed with the following parameters: [[JobParameter[name='run.id', value=1787947438958, type=class java.lang.String, identifying=true]]] and the following status: [COMPLETED] in 1s517ms
```

Aplicación de la política de omisión de registros inválidos mediante SkipPolicy, manteniendo la continuidad del Job hasta su finalización exitosa.

Evidencia 8

```
COMPARACION DE RENDIMIENTO - SPRING BATCH

transaccionesJob
1 thread : 1514 ms
3 threads : 1365 ms
5 threads : 1384 ms
Mejor configuracion: 3 threads

interesesJob
1 thread : 1112 ms
3 threads : 1135 ms
5 threads : 1170 ms
Mejor configuracion: 1 thread

estadosAnualesJob
1 thread : 1318 ms
3 threads : 1274 ms
5 threads : 1146 ms
Mejor configuracion: 5 threads
PS C:\Users\pedro\OneDrive\Escritorio\Duoc Bimestre 8\Backend-3\Semana-3\EXP1_53_Pedro_Breit\banco-batch> |
```

Comparación de tiempos de ejecución con configuraciones de 1, 3 y 5 threads para determinar la configuración más eficiente de cada Job.

Evidencia 9

```
application.properties X
EXP1_S3_Pedro_Breit > banco-batch > src > main > resources > application.properties
44 # Reglas de negocio - Intereses
45 # -----
46 banco.interes.ahorro=0.01
47 banco.interes.prestamo=0.02
48 # -----
49 # Semana 3 - Optimizacion Batch
50 # -----
51 banco.batch.chunk-size=5
52 banco.batch.threads=3
53 banco.batch.queue-capacity=10
54 banco.batch.retry-limit=3
55 banco.batch.skip-limit=1000
56 banco.batch.backoff-ms=500
```

Configuración parametrizable del tamaño de chunk, cantidad de threads y políticas de tolerancia a fallos utilizadas por los Jobs.

Evidencia 10

```
1 package cl.duoc.bancobatch.policy;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.batch.core.step.skip.SkipLimitExceededException;
6 import org.springframework.batch.core.step.skip.SkipPolicy;
7 import org.springframework.batch.item.file.FlatFileParseException;
8 import org.springframework.beans.factory.annotation.Value;
9 import org.springframework.stereotype.Component;
10
11 import cl.duoc.bancobatch.exception.DatoInvalidoException;
12
13 @Component
14 public class BancoSkipPolicy implements SkipPolicy {
15
16     private static final Logger log =
17         LoggerFactory.getLogger(BancoSkipPolicy.class);
18
19     private final long skipLimit;
20
21     public BancoSkipPolicy(
22         @Value("${banco.batch.skip-limit:100}") long skipLimit) {
23         this.skipLimit = skipLimit;
24     }
25
26     @Override
27     public boolean shouldSkip(Throwable throwable, long skipCount)
28         throws SkipLimitExceededException {
29
30         if (throwable == null) {
31             throw new IllegalArgumentException(
32                 s: "La excepción no puede ser nula"
33             );
34         }
35
36         boolean excepcionOmitible =
37             throwable instanceof DatoInvalidoException
38             || throwable instanceof FlatFileParseException;
39
40         if (!excepcionOmitible) {
41             return false;
42         }
43
44         // Spring Batch puede consultar la política con un contador negativo
45         // solo para verificar si el tipo de excepcion es omitible.
46         if (skipCount < 0) {
47             return true;
48         }
49
50         if (skipCount >= skipLimit) {
51
52             log.error(
53                 "SKIP POLICY | limite excedido={} | excepcion={}",
54                 skipLimit,
55                 throwable.getClass().getSimpleName()
56             );
57
58             throw new SkipLimitExceededException(
59                 skipLimit,
60                 throwable
61             );
62         }
63
64         log.warn(
65             "SKIP POLICY | registro omitido={} de {} | excepcion={} | motivo={}",
66             skipCount + 1,
67             skipLimit,
68             throwable.getClass().getSimpleName(),
69             throwable.getMessage()
70         );
71
72         return true;
73     }
74 }
75
```

Implementación de una política personalizada SkipPolicy para controlar y omitir registros inválidos sin interrumpir la ejecución del proceso batch.

Evidencia 11

```
J EstadosAnualesJobConfig.java X
EXP1_S3_Pedro_Breit > banco-batch > src > main > java > d > duoc > bancobatch > config > J EstadosAnualesJobConfig.j
44 public class EstadosAnualesJobConfig {
108     public Step procesarMovimientosAnualesStep(
134         .skipListener(batchSkipListener)
135         .taskExecutor(estadosAnualesTaskExecutor)
136         .listener(batchStepListener)
137
138         .build();
139     }
140
141     @Bean
142     > public JdbcCursorItemReader<EstadoCuentaAnual> estadoCuentaAnualReader(...)
204
205     @Bean
206     > public JdbcBatchItemWriter<EstadoCuentaAnual> estadoCuentaAnualWriter(...)
246
247     @Bean
248     > public FlatFileItemWriter<EstadoCuentaAnual> estadoCuentaAnualCsvWriter() { ...
278
279     @Bean
280     > public CompositeItemWriter<EstadoCuentaAnual> estadoCuentaAnualCompositeWriter(...)
291
292     @Bean
293     > public Step generarResumenAnualStep(...)
311
312     @Bean
313     > public Job estadosAnualesJob(...)
325
326     @Bean
327     public RetryPolicy bancoRetryPolicy(
328         @Value("${banco.batch.retry-limit:3}") long retryLimit,
329         @Value("${banco.batch.backoff-ms:500}") long backoffMs) {
330
331         return RetryPolicy.builder()
332             .maxRetries(retryLimit)
333             .delay(Duration.ofMillis(backoffMs))
334             .includes(TransientDataAccessException.class)
335             .build();
336     }
337 }
```

Configuración de una política de reintentos para errores transitorios de base de datos, con un máximo de 3 reintentos y un backoff de 500 ms.

Evidencia 12

```
J EstadosAnualesJobConfig.java X
EXP1_S3_Pedro_Breit > banco-batch > src > main > java > d > duoc > bancobatch > config > J EstadosAnualesJobConfig.java > Language Support for Java(TM) by Red Hat > EstadosAnualesJobConfig > procesarMovimientosAnualesStep(jobRepository, PlatformTransactionManager, SynchronizedItemStreamReader<Mov
337
338     @Bean
339     public AsyncTaskExecutor estadosAnualesTaskExecutor(
340         @Value("${banco.batch.threads:3}") int threads,
341         @Value("${banco.batch.queue-capacity:10}") int queueCapacity) {
342
343         ThreadPoolTaskExecutor taskExecutor = new ThreadPoolTaskExecutor();
344
345         taskExecutor.setCorePoolSize(threads);
346         taskExecutor.setMaxPoolSize(threads);
347         taskExecutor.setQueueCapacity(queueCapacity);
348         taskExecutor.setThreadNamePrefix("estados-anuales-");
349
350         taskExecutor.initialize();
351
352         return taskExecutor;
353     }
354 }
355 }
```

Configuración del procesamiento multihilo mediante ThreadPoolTaskExecutor, permitiendo parametrizar la cantidad de threads y la capacidad de la cola de ejecución.

Evidencia 13

```
J TransaccionProcessor.java
EXP1_S3_Pedro_Breit > banco-batch > src > main > java > d > duoc > bancobatch > processor > J TransaccionProcessor.java
18 @Component
19 public class TransaccionProcessor
20     implements ItemProcessor<TransaccionCsv, Transaccion> {
21
22     private static final List<DateTimeFormatter> FORMATOS_FECHA =
23         List.of(
24             DateTimeFormatter.ofPattern(pattern: "yyyy-MM-dd"),
25             DateTimeFormatter.ofPattern(pattern: "yyyy/MM/dd"),
26             DateTimeFormatter.ofPattern(pattern: "dd-MM-yyyy"),
27             DateTimeFormatter.ofPattern(pattern: "dd/MM/yyyy")
28         );
29
30     @Override
31     public Transaccion process(TransaccionCsv item) { ...
32
33     private void validarId(TransaccionCsv item) {
34
35         if (item.getId() == null || item.getId() <= 0) {
36             throw new DatoInvalidoException(
37                 "ID de transaccion invalido: "
38                 + item.getId()
39             );
40         }
41     }
42
43     private BigDecimal normalizarMonto(
44         TransaccionCsv item) {
45
46         BigDecimal monto = item.getMonto();
47
48         if (monto == null) {
49             throw new DatoInvalidoException(
50                 "Monto nulo en transaccion ID "
51                 + item.getId()
52             );
53         }
54
55         if (monto.compareTo(BigDecimal.ZERO) == 0) {
56             throw new DatoInvalidoException(
57                 "Monto no puede ser cero. ID: "
58                 + item.getId()
59             );
60         }
61
62         return monto.abs();
63     }
64 }

J TransaccionProcessor.java
EXP1_S3_Pedro_Breit > banco-batch > src > main > java > d > duoc > bancobatch > processor > J TransaccionProcessor.java
19 public class TransaccionProcessor
20
21     private String normalizarTipo(String tipo) {
22
23         if (tipo == null || tipo.isBlank()) {
24             throw new DatoInvalidoException(
25                 message: "Tipo de transaccion vacio"
26             );
27         }
28
29         String tipoNormalizado =
30             quitarAcentos(tipo)
31                 .trim()
32                 .toLowerCase(Locale.ROOT);
33
34         if (!tipoNormalizado.equals("@Object: \"debito\")
35             && !tipoNormalizado.equals("@Object: \"credito\") ) {
36
37             throw new DatoInvalidoException(
38                 "Tipo de transaccion invalido: "
39                 + tipo
40             );
41         }
42
43         return tipoNormalizado;
44     }
45
46     private LocalDate convertirFecha(String fecha) {
47
48         if (fecha == null || fecha.isBlank()) {
49             throw new DatoInvalidoException(
50                 message: "Fecha de transaccion vacia"
51             );
52         }
53
54         String fechaLimpia = fecha.trim();
55
56         for (DateTimeFormatter formato : FORMATOS_FECHA) {
57             try {
58                 return LocalDate.parse(
59                     fechaLimpia,
60                     formato
61                 );
62             } catch (DateTimeParseException ignored) {
63                 // Se prueba el siguiente formato.
64             }
65         }
66
67         throw new DatoInvalidoException(
68             "Formato de fecha invalido: "
69             + fecha
70         );
71     }
72
73     private String quitarAcentos(String valor) {
74
75         return Normalizer
76             .normalize(
77                 valor,
78                 Normalizer.Form.NFD
79             )
80             .replaceAll(regex: "\\p{M}", replacement: "");
81     }
82 }
```

Implementación de validaciones y transformaciones mediante ItemProcessor para normalizar montos, tipos de transacción y formatos de fecha.

Evidencia 14

```
1 package cl.duoc.bancobatch.processor;
2
3 import java.math.BigDecimal;
4 import java.math.RoundingMode;
5 import java.text.Normalizer;
6 import java.util.Locale;
7
8 import org.springframework.batch.item.ItemProcessor;
9 import org.springframework.beans.factory.annotation.Value;
10 import org.springframework.stereotype.Component;
11
12 import cl.duoc.bancobatch.exception.DatoInvalidoException;
13 import cl.duoc.bancobatch.model.InteresCalculado;
14 import cl.duoc.bancobatch.model.InteresCsv;
15
16 @Component
17 public class InteresProcessor
18     implements ItemProcessor<InteresCsv, InteresCalculado> {
19
20     private final BigDecimal tasaAhorro;
21     private final BigDecimal tasaPrestamo;
22
23     public InteresProcessor(
24         @Value("${banco.interes.ahorro}") BigDecimal tasaAhorro,
25         @Value("${banco.interes.prestamo}") BigDecimal tasaPrestamo) {
26
27         this.tasaAhorro = tasaAhorro;
28         this.tasaPrestamo = tasaPrestamo;
29     }
30
31     @Override
32     public InteresCalculado process(InteresCsv item) {
33
34         validarCuenta(item);
35
36         String nombreNormalizado =
37             normalizarNombre(
38                 item.getNombre(),
39                 item.getCuentaId()
40             );
41
42         String tipoNormalizado =
43             normalizarTipo(item.getTipo());
44
45         BigDecimal saldoNormalizado =
46             normalizarSaldo(
47                 item.getSaldo(),
48                 item.getCuentaId()
49             );
50
51         Integer edadNormalizada =
52             validarEdad(
53                 item.getEdad(),
54                 item.getCuentaId()
55             );
56
57         BigDecimal tasa =
58             obtenerTasa(tipoNormalizado);
59
151
152     private Integer validarEdad(
153         Integer edad,
154         Long cuentaId) {
155
156         if (edad == null) {
157
158             throw new DatoInvalidoException(
159                 "Edad nula en cuenta ID "
160                     + cuentaId
161             );
162         }
163
164         if (edad < 0 || edad > 120) {
165
166             throw new DatoInvalidoException(
167                 "Edad fuera de rango. Cuenta ID: "
168                     + cuentaId
169                     + ", edad: "
170                     + edad
171             );
172         }
173
174         return edad;
175     }
176
177     private String normalizarTipo(String tipo) {
178
179         if (tipo == null || tipo.isBlank()) {
180
181             throw new DatoInvalidoException(
182                 message: "Tipo de cuenta vacio"
183             );
184         }
185
186         String tipoNormalizado =
187             quitarAcentos(tipo)
188                 .trim()
189                 .toLowerCase(Locale.ROOT);
190
191         if (tipoNormalizado.equals(anObject: "hipoteca")) {
192             return "prestamo";
193         }
194
195         if (!tipoNormalizado.equals(anObject: "ahorro")
196             && !tipoNormalizado.equals(anObject: "prestamo")) {
197
198             throw new DatoInvalidoException(
199                 "Tipo de cuenta no permitido: "
200                     + tipo
201             );
202         }
203
204         return tipoNormalizado;
205     }
206
207 > private BigDecimal obtenerTasa(...)
225
226 > private String quitarAcentos(...)
236
237 }
```

Validación, normalización y cálculo de intereses mediante ItemProcessor, aplicando reglas de negocio para cuentas de ahorro y préstamo.

Evidencia 15

```
18 @Component
19 public class MovimientoAnualProcessor
20     implements ItemProcessor<MovimientoAnualCsv, MovimientoAnual> {
21
22     private static final List<DateTimeFormatter> FORMATOS_FECHA =
23         List.of(
24             DateTimeFormatter.ofPattern(pattern: "yyyy-MM-dd"),
25             DateTimeFormatter.ofPattern(pattern: "yyyy/MM/dd"),
26             DateTimeFormatter.ofPattern(pattern: "dd-MM-yyyy"),
27             DateTimeFormatter.ofPattern(pattern: "dd/MM/yyyy")
28         );
29
30     @Override
31     public MovimientoAnual process(MovimientoAnualCsv item) {
32
33         validarCuenta(item);
34
35         LocalDate fechaNormalizada =
36             convertirFecha(item.getFecha());
37
38         String transaccionNormalizada =
39             normalizarTransaccion(item.getTransaccion());
40
41         BigDecimal montoNormalizado =
42             normalizarMonto(
43                 item.getMonto(),
44                 transaccionNormalizada,
45                 item.getCuentaId()
46             );
47
48         String descripcionNormalizada =
49             normalizarDescripcion(item.getDescripcion());
50
51         return new MovimientoAnual(
52             item.getCuentaId(),
53             fechaNormalizada,
54             transaccionNormalizada,
55             montoNormalizado,
56             descripcionNormalizada
57         );
58     }
59
60     private void validarCuenta(MovimientoAnualCsv item) { ...
61
62     private String normalizarTransaccion(
63         String transaccion) {
64
65         if (transaccion == null
66             || transaccion.isBlank()) {
67
68             throw new DatoInvalidoException(
69                 message: "Tipo de transaccion vacio"
70             );
71         }
72
73         String valor =
74             quitarAcentos(transaccion)
75                 .trim()
76                 .toLowerCase(Locale.ROOT);
77
78         if (!valor.equals(anObject: "deposito")
79             && !valor.equals(anObject: "retiro")
80             && !valor.equals(anObject: "compra")) {
81
82             throw new DatoInvalidoException(
83                 "Tipo de transaccion no permitido: "
84                     + transaccion
85             );
86         }
87
88         return valor;
89     }
90
91     private BigDecimal normalizarMonto(
92         BigDecimal monto,
93         String transaccion,
94         Long cuentaId) {
95
96         if (monto == null) {
97             throw new DatoInvalidoException(
98                 "Monto nulo en cuenta ID "
99                     + cuentaId
100             );
101         }
102
103         if (monto.compareTo(BigDecimal.ZERO) == 0) {
104             throw new DatoInvalidoException(
105                 "Monto no puede ser cero. Cuenta ID: "
106                     + cuentaId
107             );
108         }
109
110         BigDecimal magnitud = monto.abs();
111
112         if (transaccion.equals(anObject: "deposito")) {
113             return magnitud;
114         }
115
116         return magnitud.negate();
117     }
118
119     private String normalizarDescripcion( ...
120
121     private LocalDate convertirFecha(
122         String fecha) {
123
124         if (fecha == null
125             || fecha.isBlank()) {
126
127             throw new DatoInvalidoException(
128                 message: "Fecha de movimiento vacia"
129             );
130         }
131
132         for (DateTimeFormatter formatter : FORMATOS_FECHA) {
133             try {
134                 return LocalDate.parse(fecha, formatter);
135             } catch (ParseException e) {
136                 continue;
137             }
138         }
139
140         throw new DatoInvalidoException(
141             "Fecha no es compatible con ningun formato de fecha"
142         );
143     }
144 }
```

Validación y normalización de movimientos anuales mediante ItemProcessor, considerando tipos de transacción, montos y múltiples formatos de fecha.

Evidencia 16

```
PS C:\Users\pedro\OneDrive\Escritorio\Banco Batch\Backend-3\Semana-3\EXPI_53_Pedro_Breit\banco-batch> mvnw.cmd clean compile
[INFO] Scanning for projects...
[INFO]
[INFO] <-----> c:\duc\banco-batch <----->
[INFO] Building banco-batch 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO]
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- class:3.5.0:clean (default-clean) @ banco-batch ---
[INFO] Deleting C:\Users\pedro\OneDrive\Escritorio\Banco Batch\Backend-3\Semana-3\EXPI_53_Pedro_Breit\banco-batch\target
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ banco-batch ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 4 resources from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.10.1:compile (default-compile) @ banco-batch ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 19 source files with javac [debug parameters release 21] to target/classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 2.103 s
[INFO] Finished at: 2026-08-28T16:49:28-04:00
[INFO]
PS C:\Users\pedro\OneDrive\Escritorio\Banco Batch\Backend-3\Semana-3\EXPI_53_Pedro_Breit\banco-batch >
```

Compilación exitosa del proyecto Spring Batch mediante Maven, verificando la integridad final del código fuente.